

Smart Link Manager - Day 1 Detailed Notes

Project Goal

Build a Smart Link Manager that can: 1. Accept a URL from the user. 2. Generate a short URL. 3. Generate AI summaries and tags. 4. Store links in a database. 5. Show saved links and analytics. We intentionally started with the frontend because users interact with the UI first. Before AI, databases, or backend logic, we need a working interface.

Step 1: Creating the Project Folder

Commands: `mkdir smart-link-manager` `cd smart-link-manager` Why? A project folder acts as the root of the application. Every file related to the project lives inside it. This keeps the code organized and makes GitHub, deployment, and maintenance easier.

Step 2: Opening VS Code

Command: `code .` The dot means 'current folder'. VS Code opens the project directory as a workspace. This allows us to manage all project files from a single place.

Step 3: Creating the React Frontend

Command: `npm create vite@latest frontend` Why React? The frontend is responsible for everything the user sees: - Input boxes - Buttons - Summaries - Saved links Why Vite? Vite is a project generator and development tool. It creates the folder structure, development server, and configuration required to run React applications.

Step 4: npm install

Command: `npm install` `package.json` contains a list of dependencies needed by the project. `npm install` reads `package.json` and downloads all required libraries. Without this step, React cannot run.

Frontend Folder Structure

Important folders/files: `src/` Contains application code. `public/` Contains static assets. `package.json` Stores project metadata and dependency information. `App.jsx` Main React component we worked on today.

What is a React Component?

Code: `function App() {}` A component is a reusable UI block. Mental model: Java Class -> React Component The `App` component is responsible for generating the UI displayed on screen.

Replacing the Default Vite Page

We replaced the template page with: Smart Link Manager AI-powered URL shortening and summarization Why? A project should display our application, not the Vite demo page.

Adding an Input Box and Button

We added: Purpose: Users need a place to enter URLs. The button starts the generation process. This mirrors the future flow: User enters URL -> Click Generate -> Get summary.

React State

Code: `const [url, setUrl] = useState("");` State = Data React remembers. url: Current stored URL. setUrl: Function used to update the URL. Why state? The application must remember what the user typed.

What is useState?

useState creates state. Example: `const [url, setUrl] = useState("");` Initially: url = "" Later: `setUrl("https://leetcode.com")` React updates the stored value and refreshes the UI.

Controlled Inputs

Code: `value={url} onChange={(e) => setUrl(e.target.value)}` value={url} Makes React control the input. onChange Runs whenever the user types. e.target.value Contains the current text inside the input box. Why important? React becomes the source of truth instead of the HTML element.

Data Flow Learned Today

User Types ↓ onChange Event ↓ setUrl() ↓ url State Updates ↓ React Re-renders ↓ UI Updates
This is one of the most important React concepts.

Conditional Rendering

Code: `{summary} && {summary}` Meaning: Show the paragraph only when summary contains data. Why useful? We don't want empty summary boxes on the screen. Same idea will later be used for: - Error messages - Loading indicators - Generated short URLs - AI tags

Validation

State: `const [error, setError] = useState("");` Logic: If URL is empty: - Show error - Clear summary
Why? Real applications must validate user input before processing requests.

Loading State

State: `const [loading, setLoading] = useState(false);` Purpose: Tell users that work is being performed. Future flow: Generate ↓ Backend Request ↓ Gemini API ↓ Summary Without loading, users think the app is frozen.

Re-rendering

This is the most important concept from Day 1. Whenever state changes: `setUrl(...)` `setSummary(...)` `setError(...)` `setLoading(...)` React re-runs the component and updates the UI. Mental model: State Changes ↓ React Re-renders ↓ UI Changes NOT: UI Changes ↓ State Changes

Why Today's Work Was Crucial

Although the project is only around 15-20% complete, today's work built the foundation. Completed:
- React setup - Component creation - State management - Event handling - Conditional rendering - Validation - Loading state These concepts will be reused in: - Backend integration - Gemini API integration - MongoDB storage - URL shortening - AI summaries - AI tags Most future features are built on top of today's concepts.

Interview Questions You Should Be Able to Answer

1. What is React? 2. What is a component? 3. What is state? 4. Why use useState? 5. What is onChange? 6. What is e.target.value? 7. What is conditional rendering? 8. What is a controlled component? 9. What is re-rendering? 10. Why do we use loading states?